МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

КАФЕДРА ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ



Лабораторная работа №2

по дисциплине «Технологии разработки программного обеспечения»

на тему

«Разработка класса для представления структуры данных на Java»

Группа: АММ26

Факультет: АВТФ

Студент: Арещенко Е.А.

Преподаватель: Токарев В.Г.

Новосибирск 2026

Оглавление

1. Задание на лабораторную работу 3

2. Исходные данные 3

3. Ход работы 3

3.1. Структура двусвязного списка 4

3.2. Добавление в конец и вставка по логическому номеру 5

3.3. Получение по логическому номеру 6

3.4. Удаление по логическому номеру 6

3.5. Итератор forEach 7

3.6. Сортировка рекурсивным слиянием 8

3.6.1. Рекурсивное разделение 8

3.6.2. Слияние упорядоченных частей 9

3.6.3. Восстановление обратных ссылок 10

3.7. Обработка ошибок 10

3.8. Пример работы разработанного ПО 10

Заключение 12

Приложение А 14

1. Задание на лабораторную работу

Портировать одну из структур данных, приведённых в *cprog*. Набор операций для структуры данных:

- добавление в конец, получение, вставка и удаление по логическому номеру (индексу). Логический номер – порядковый номер хранимого элемента в соответствии с последовательностью обхода структуры данных (перебора элементов);
- итератор *forEach* – для *Java*: интерфейс обратного вызова с методом *void toDo(T v)*; итератор получает ссылку на интерфейсный объект (анонимный класс), обходит структуру данных и вызывает метод для каждого выбранного элемента;
- сортировка эффективным алгоритмом сложности O(N·log N).

Элемент структуры данных хранит целое число. В *main* написать тестовую последовательность создания объекта структуры данных и выполнения над ним операций с трассировкой результатов.

2. Исходные данные

Согласно варианту задания №8 на лабораторную работу имеем:

- Структура данных: двусвязный разомкнутый список;
- Алгоритм сортировки: рекурсивное слияние;
- Тип хранимых данных: целое число (*int*);
- Язык программирования: *Java* (JDK 21).

3. Ход работы

Для решения задачи были разработаны: класс *DoublyLinkedList*, реализующий двусвязный разомкнутый список, интерфейс обратного вызова *ToDo<T>* для итератора и класс *Main* с тестовой последовательностью. Полный исходный код приведён в Приложении А.

3.1. Структура двусвязного списка

Двусвязный список состоит из элементов (узлов), каждый из которых хранит значение и две ссылки: на предыдущий и на следующий элемент. Список называется разомкнутым, так как у первого элемента ссылка на предыдущий равна *null*, а у последнего – ссылка на следующий равна *null*, то есть список не замкнут в кольцо.

Узел списка описан вложенным статическим классом *Node* с полями:

- *int value* – хранимое целое число;
- *Node prev* – ссылка на предыдущий элемент;
- *Node next* – ссылка на следующий элемент.

Класс *DoublyLinkedList* содержит поля:

- *Node head* – ссылка на первый элемент списка;
- *Node tail* – ссылка на последний элемент списка;
- *int size* – количество элементов.

Хранение ссылки на последний элемент позволяет добавлять элементы в конец за O(1), а наличие обратных ссылок – обходить список в обоих направлениях и удалять элемент без поиска его предшественника (листинг 1).

```
private static class Node {
    int value;
    Node prev;
    Node next;

    Node(int value) {
        this.value = value;
    }
}

private Node head;
private Node tail;
private int size;
```

Листинг 1. Узел списка и поля класса DoublyLinkedList

Класс *Node* объявлен как *private static*: он является деталью реализации и недоступен снаружи, а модификатор *static* означает, что узел не хранит скрытую ссылку на объект списка, что экономит память.

3.2. Добавление в конец и вставка по логическому номеру

Метод *addLast(int value)* создаёт новый узел и присоединяет его после *tail*. Если список пуст, новый узел становится одновременно первым и последним. Аналогично реализован метод *addFirst(int value)* для добавления в начало.

Метод *insert(int index, int value)* вставляет элемент так, чтобы он получил логический номер *index*. Допустимы значения от 0 до *size* включительно: вставка в позицию 0 сводится к *addFirst*, в позицию *size* – к *addLast*. В остальных случаях находится узел, стоящий на позиции *index*, и новый узел встраивается перед ним: изменяются четыре ссылки – две у нового узла и по одной у его соседей (листинг 2).

```
public void addLast(int value) {
    Node node = new Node(value);
    if (tail == null) {
        head = node;
    } else {
        tail.next = node;
        node.prev = tail;
    }
    tail = node;
    size++;
}

public void insert(int index, int value) {
    if (index < 0 || index > size) {
        throw new IndexOutOfBoundsException("Неверный индекс вставки: " +
index + ", размер: " + size);
    }
    if (index == 0) {
        addFirst(value);
        return;
    }
    if (index == size) {
        addLast(value);
        return;
    }
    Node next = nodeAt(index);
    Node prev = next.prev;
    Node node = new Node(value);
    node.prev = prev;
    node.next = next;
    prev.next = node;
    next.prev = node;
    size++;
}
```

Листинг 2. Добавление в конец и вставка по логическому номеру

3.3. Получение по логическому номеру

Логический номер элемента – это его порядковый номер при обходе списка от *head* к *tail*, начиная с нуля. Поиск узла по номеру выполняет закрытый метод *nodeAt(int index)*, который используется методами *get*, *set*, *insert* и *remove*.

Двусвязность списка позволяет ускорить поиск: если номер находится в первой половине списка, обход ведётся от начала по ссылкам *next*, иначе – от конца по ссылкам *prev*. Таким образом, количество шагов не превышает *size*/2 (листинг 3).

```
private Node nodeAt(int index) {
    if (index < 0 || index >= size) {
        throw new IndexOutOfBoundsException("Неверный индекс: " + index + ",
размер: " + size);
    }
    Node cur;
    if (index < size / 2) {
        cur = head;
        for (int i = 0; i < index; i++) {
            cur = cur.next;
        }
    } else {
        cur = tail;
        for (int i = size - 1; i > index; i--) {
            cur = cur.prev;
        }
    }
    return cur;
}
```

Листинг 3. Поиск узла по логическому номеру

3.4. Удаление по логическому номеру

Метод *remove(int index)* находит удаляемый узел и «перекидывает» ссылки его соседей друг на друга, исключая узел из цепочки. Отдельно обрабатываются случаи, когда удаляется первый элемент (изменяется *head*) или последний (изменяется *tail*). У удалённого узла ссылки обнуляются, после чего он становится недоступен и освобождается сборщиком мусора. Метод возвращает значение удалённого элемента (листинг 4).

```
public int remove(int index) {
    Node node = nodeAt(index);
    if (node.prev == null) {
        head = node.next;
    } else {
        node.prev.next = node.next;
    }
    if (node.next == null) {
        tail = node.prev;
    } else {
        node.next.prev = node.prev;
    }
    node.prev = null;
    node.next = null;
    size--;
    return node.value;
}
```

Листинг 4. Удаление по логическому номеру

Таблица 1. Трудоёмкость операций над списком

Операция	Метод	Сложность
Добавление в конец / начало	addLast, addFirst	O(1)
Получение по номеру	get	O(N)
Вставка по номеру	insert	O(N)
Удаление по номеру	remove	O(N)
Обход	forEach	O(N)
Сортировка	sort	O(N·log N)

Для операций по номеру основное время тратится на поиск узла; само изменение ссылок выполняется за O(1).

3.5. Итератор forEach

Итератор реализован через интерфейс обратного вызова *ToDo<T>* с единственным методом *void toDo(T v)*. Метод *forEach* получает ссылку на объект, реализующий этот интерфейс, проходит список от первого элемента к последнему и для каждого элемента вызывает *toDo*. Дополнительно реализован *forEachReverse* – обход от последнего элемента к первому по ссылкам *prev* (листинг 5).

```
public interface ToDo<T> {
    void toDo(T v);
}

public void forEach(ToDo<Integer> action) {
    for (Node cur = head; cur != null; cur = cur.next) {
        action.toDo(cur.value);
    }
}
```

Листинг 5. Интерфейс обратного вызова и итератор forEach

При вызове итератора в *main* интерфейсный объект создаётся анонимным классом. Такой подход позволяет задавать произвольное действие над элементами, не изменяя класс списка: вывод на экран, подсчёт суммы, отбор чётных элементов (листинг 6).

```
final int[] sum = {0};
list.forEach(new ToDo<Integer>() {
    @Override
    public void toDo(Integer v) {
        sum[0] += v;
    }
});
System.out.println("Сумма элементов: " + sum[0]);
```

Листинг 6. Вызов итератора с анонимным классом

Сумма накапливается в массиве из одного элемента, поскольку анонимный класс может обращаться только к локальным переменным, значение которых не изменяется (*effectively final*), а содержимое массива изменять можно.

3.6. Сортировка рекурсивным слиянием

Сортировка слиянием (*merge sort*) хорошо подходит для связных списков: в отличие от массивов, ей не требуется произвольный доступ к элементам и дополнительная память под копию данных – узлы просто перецепляются. Алгоритм состоит из трёх этапов.

3.6.1. Рекурсивное разделение

Метод *mergeSort* делит список пополам и рекурсивно сортирует каждую половину. Середина находится методом «быстрого и медленного указателя» (*middleOf*): указатель *slow* сдвигается на один узел, а *fast* – на два; когда *fast* доходит до конца, *slow* оказывается в середине. После этого ссылка

next среднего узла обнуляется, и список распадается на две независимые части. Рекурсия заканчивается на списках из 0 или 1 элемента, которые уже упорядочены (листинг 7).

```
private Node mergeSort(Node first) {
    if (first == null || first.next == null) {
        return first;
    }
    Node middle = middleOf(first);
    Node second = middle.next;
    middle.next = null;
    return merge(mergeSort(first), mergeSort(second));
}

private Node middleOf(Node first) {
    Node slow = first;
    Node fast = first.next;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}
```

Листинг 7. Рекурсивное разделение списка

3.6.2. Слияние упорядоченных частей

Метод *merge* объединяет два упорядоченных списка в один. Используется фиктивный узел *dummy*, к которому поочерёдно присоединяется меньший из головных элементов двух списков. Когда один из списков заканчивается, остаток второго присоединяется целиком. При равных значениях первым берётся элемент из левой части, поэтому сортировка устойчива. Слияние выполнено циклом, а не рекурсией, чтобы глубина стека не зависела от длины списка (листинг 8).

```
private Node merge(Node a, Node b) {
    Node dummy = new Node(0);
    Node last = dummy;
    while (a != null && b != null) {
        if (a.value <= b.value) {
            last.next = a;
            a = a.next;
        } else {
            last.next = b;
            b = b.next;
        }
        last = last.next;
    }
    last.next = (a != null) ? a : b;
    return dummy.next;
}
```

Листинг 8. Слияние двух упорядоченных списков

3.6.3. Восстановление обратных ссылок

Во время разделения и слияния изменяются только ссылки *next*, поэтому после сортировки метод *sort* одним проходом восстанавливает ссылки *prev* и находит новый последний элемент *tail* (листинг 9).

```
public void sort() {
    head = mergeSort(head);
    Node prev = null;
    for (Node cur = head; cur != null; cur = cur.next) {
        cur.prev = prev;
        prev = cur;
    }
    tail = prev;
}
```

Листинг 9. Главный метод сортировки

Глубина рекурсии разделения равна $\log_2 N$, на каждом уровне рекурсии суммарно обрабатывается N элементов, поэтому общая трудоёмкость сортировки составляет $O(N \cdot \log N)$. Для списка из 100 000 элементов глубина рекурсии – около 17 вызовов.

3.7. Обработка ошибок

При обращении к несуществующему логическому номеру (отрицательному или не меньшему размера списка, а для вставки – большему размера) генерируется исключение *IndexOutOfBoundsException* с указанием неверного индекса и текущего размера списка.

3.8. Пример работы разработанного ПО

В классе *Main* выполняется тестовая последовательность с трассировкой результатов каждой операции: создание списка и добавление элементов в конец, получение, вставка и удаление по логическому номеру, обход итератором в прямом и обратном направлении, сортировка, проверка граничных случаев и ошибок. Результаты приведены на рисунках 1–3.

```
=== Создание списка ===
Пустой список: [], размер = 0
addLast(42): [42]
addLast(7): [42 <-> 7]
addLast(19): [42 <-> 7 <-> 19]
addLast(3): [42 <-> 7 <-> 19 <-> 3]
addLast(88): [42 <-> 7 <-> 19 <-> 3 <-> 88]
addLast(25): [42 <-> 7 <-> 19 <-> 3 <-> 88 <-> 25]
addLast(61): [42 <-> 7 <-> 19 <-> 3 <-> 88 <-> 25 <-> 61]
addLast(14): [42 <-> 7 <-> 19 <-> 3 <-> 88 <-> 25 <-> 61 <-> 14]
Размер = 8

=== Получение по логическому номеру ===
get(0) = 42
get(3) = 3
get(7) = 14

=== Вставка по логическому номеру ===
insert(0, 100):  [100 <-> 42 <-> 7 <-> 19 <-> 3 <-> 88 <-> 25 <-> 61 <-> 14]
insert(4, 200):  [100 <-> 42 <-> 7 <-> 19 <-> 200 <-> 3 <-> 88 <-> 25 <-> 61 <-> 14]
insert(10, 300): [100 <-> 42 <-> 7 <-> 19 <-> 200 <-> 3 <-> 88 <-> 25 <-> 61 <-> 14 <-> 300]
Размер = 11
```

Рисунок 1. Создание списка, получение и вставка по логическому номеру

```
=== Удаление по логическому номеру ===
remove(0) = 100  -> [42 <-> 7 <-> 19 <-> 200 <-> 3 <-> 88 <-> 25 <-> 61 <-> 14 <-> 300]
remove(3) = 200  -> [42 <-> 7 <-> 19 <-> 3 <-> 88 <-> 25 <-> 61 <-> 14 <-> 300]
remove(8) = 300  -> [42 <-> 7 <-> 19 <-> 3 <-> 88 <-> 25 <-> 61 <-> 14]
Размер = 8

=== Итератор forEach (анонимные классы) ===
Прямой обход:   42 7 19 3 88 25 61 14
Обратный обход: 14 61 25 88 3 19 7 42
Сумма элементов: 259
Чётные элементы: 42 88 14
```

Рисунок 2. Удаление по логическому номеру и итератор forEach

```
=== Сортировка рекурсивным слиянием ===
До сортировки:    [42 <-> 7 <-> 19 <-> 3 <-> 88 <-> 25 <-> 61 <-> 14]
После сортировки: [3 <-> 7 <-> 14 <-> 19 <-> 25 <-> 42 <-> 61 <-> 88]
Обратный обход:   88 61 42 25 19 14 7 3
Упорядочен: true

=== Сортировка списка из 100000 случайных чисел ===
Время сортировки: 42,1 мс
Упорядочен: true
Совпадает с Arrays.sort: true

=== Граничные случаи ===
Сортировка из 1 элемента: [5]
remove(0) = 5, список: [], пуст: true
insert(0, 9) в пустой список: [9]
Сортировка с повторами: [1 <-> 1 <-> 2 <-> 3 <-> 3 <-> 3]

=== Ошибки ===
Ошибка: Неверный индекс: 100, размер: 8
Ошибка: Неверный индекс вставки: -1, размер: 8
Ошибка: Неверный индекс: 0, размер: 0

После clear(): [], размер = 0
```

Рисунок 3. Сортировка, граничные случаи и обработка ошибок

Для проверки корректности сортировки список из 100 000 случайных чисел был отсортирован разработанным методом, а копия тех же данных – стандартным методом *Arrays.sort*. Результаты совпали. Обратный обход отсортированного списка выдаёт элементы в порядке убывания, что подтверждает корректность восстановления ссылок *prev*.

Заключение

В ходе выполнения лабораторной работы был разработан и протестирован класс *DoublyLinkedList*, реализующий структуру данных «двусвязный разомкнутый список» с элементами, хранящими целые числа.

Реализованы операции добавления в конец, получения, вставки и удаления по логическому номеру; за счёт двусвязности поиск элемента ведётся с ближайшего конца списка. Итератор *forEach* реализован через интерфейс обратного вызова *ToDo<T>*, которому в тестовой программе передаются анонимные классы.

Сортировка выполнена алгоритмом рекурсивного слияния со сложностью O(N·log N). Алгоритм не копирует данные, а перестраивает связи между узлами, что делает его естественным выбором для связного списка. Корректность работы подтверждена сравнением с библиотечной сортировкой на 100 000 элементов.

Приложение А

Исходный код класса DoublyLinkedList (файл DoublyLinkedList.java)

```
public class DoublyLinkedList {

    private static class Node {
        int value;
        Node prev;
        Node next;

        Node(int value) {
            this.value = value;
        }
    }

    private Node head;
    private Node tail;
    private int size;

    public DoublyLinkedList() {
    }

    public DoublyLinkedList(int... values) {
        for (int v : values) {
            addLast(v);
        }
    }

    public int size() {
        return size;
    }

    public boolean isEmpty() {
        return size == 0;
    }

    public void clear() {
        Node cur = head;
        while (cur != null) {
            Node next = cur.next;
            cur.prev = null;
            cur.next = null;
            cur = next;
        }
        head = null;
        tail = null;
        size = 0;
    }

    public void addLast(int value) {
        Node node = new Node(value);
        if (tail == null) {
            head = node;
        } else {
            tail.next = node;
            node.prev = tail;
        }
        tail = node;
        size++;
    }

    public void addFirst(int value) {
```

```
        Node node = new Node(value);
        if (head == null) {
            tail = node;
        } else {
            head.prev = node;
            node.next = head;
        }
        head = node;
        size++;
    }

    public int get(int index) {
        return nodeAt(index).value;
    }

    public void set(int index, int value) {
        nodeAt(index).value = value;
    }

    public void insert(int index, int value) {
        if (index < 0 || index > size) {
            throw new IndexOutOfBoundsException("Неверный индекс вставки: " + index
+ ", размер: " + size);
        }
        if (index == 0) {
            addFirst(value);
            return;
        }
        if (index == size) {
            addLast(value);
            return;
        }
        Node next = nodeAt(index);
        Node prev = next.prev;
        Node node = new Node(value);
        node.prev = prev;
        node.next = next;
        prev.next = node;
        next.prev = node;
        size++;
    }

    public int remove(int index) {
        Node node = nodeAt(index);
        if (node.prev == null) {
            head = node.next;
        } else {
            node.prev.next = node.next;
        }
        if (node.next == null) {
            tail = node.prev;
        } else {
            node.next.prev = node.prev;
        }
        node.prev = null;
        node.next = null;
        size--;
        return node.value;
    }

    public void forEach(ToDo<Integer> action) {
        for (Node cur = head; cur != null; cur = cur.next) {
            action.toDo(cur.value);
        }
    }

    public void forEachReverse(ToDo<Integer> action) {
```

```
        for (Node cur = tail; cur != null; cur = cur.prev) {
            action.toDo(cur.value);
        }
    }

    public void sort() {
        head = mergeSort(head);
        Node prev = null;
        for (Node cur = head; cur != null; cur = cur.next) {
            cur.prev = prev;
            prev = cur;
        }
        tail = prev;
    }

    private Node mergeSort(Node first) {
        if (first == null || first.next == null) {
            return first;
        }
        Node middle = middleOf(first);
        Node second = middle.next;
        middle.next = null;
        return merge(mergeSort(first), mergeSort(second));
    }

    private Node middleOf(Node first) {
        Node slow = first;
        Node fast = first.next;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }

    private Node merge(Node a, Node b) {
        Node dummy = new Node(0);
        Node last = dummy;
        while (a != null && b != null) {
            if (a.value <= b.value) {
                last.next = a;
                a = a.next;
            } else {
                last.next = b;
                b = b.next;
            }
            last = last.next;
        }
        last.next = (a != null) ? a : b;
        return dummy.next;
    }

    public boolean isSorted() {
        for (Node cur = head; cur != null && cur.next != null; cur = cur.next) {
            if (cur.value > cur.next.value) {
                return false;
            }
        }
        return true;
    }

    public int[] toArray() {
        int[] res = new int[size];
        int i = 0;
        for (Node cur = head; cur != null; cur = cur.next) {
            res[i++] = cur.value;
        }
```

```
        return res;
    }


    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder("[");
        for (Node cur = head; cur != null; cur = cur.next) {
            sb.append(cur.value);
            if (cur.next != null) {
                sb.append(" <-> ");
            }
        }
        return sb.append("]").toString();
    }


    private Node nodeAt(int index) {
        if (index < 0 || index >= size) {
            throw new IndexOutOfBoundsException("Неверный индекс: " + index + ",
размер: " + size);
        }
        Node cur;
        if (index < size / 2) {
            cur = head;
            for (int i = 0; i < index; i++) {
                cur = cur.next;
            }
        } else {
            cur = tail;
            for (int i = size - 1; i > index; i--) {
                cur = cur.prev;
            }
        }
        return cur;
    }
}
```

Исходный код интерфейса ToDo (файл ToDo.java)

```
public interface ToDo<T> {
    void toDo(T v);
}
```

Исходный код класса Main (файл Main.java)

```
import java.util.Arrays;
import java.util.Random;


public class Main {


    public static void main(String[] args) {
        System.out.println("=== Создание списка ===");
        DoublyLinkedList list = new DoublyLinkedList();
        System.out.println("Пустой список: " + list + ", размер = " + list.size());
        for (int v : new int[]{42, 7, 19, 3, 88, 25, 61, 14}) {
            list.addLast(v);
            System.out.println("addLast(" + v + "): " + list);
        }
        System.out.println("Размер = " + list.size());


        System.out.println("\n=== Получение по логическому номеру ===");
        System.out.println("get(0) = " + list.get(0));
        System.out.println("get(3) = " + list.get(3));
        System.out.println("get(" + (list.size() - 1) + ") = " +
list.get(list.size() - 1));
```

```
        System.out.println("\n=== Вставка по логическому номеру ===");
        list.insert(0, 100);
        System.out.println("insert(0, 100):  " + list);
        list.insert(4, 200);
        System.out.println("insert(4, 200):  " + list);
        list.insert(list.size(), 300);
        System.out.println("insert(" + (list.size() - 1) + ", 300): " + list);
        System.out.println("Размер = " + list.size());


        System.out.println("\n=== Удаление по логическому номеру ===");
        System.out.println("remove(0) = " + list.remove(0) + "  -> " + list);
        System.out.println("remove(3) = " + list.remove(3) + "  -> " + list);
        System.out.println("remove(" + (list.size() - 1) + ") = " +
list.remove(list.size() - 1) + "  -> " + list);
        System.out.println("Размер = " + list.size());


        System.out.println("\n=== Итератор forEach (анонимные классы) ===");
        System.out.print("Прямой обход:   ");
        list.forEach(new ToDo<Integer>() {
            @Override
            public void toDo(Integer v) {
                System.out.print(v + " ");
            }
        });
        System.out.print("\nОбратный обход: ");
        list.forEachReverse(new ToDo<Integer>() {
            @Override
            public void toDo(Integer v) {
                System.out.print(v + " ");
            }
        });
        System.out.println();


        final int[] sum = {0};
        list.forEach(new ToDo<Integer>() {
            @Override
            public void toDo(Integer v) {
                sum[0] += v;
            }
        });
        System.out.println("Сумма элементов: " + sum[0]);


        System.out.print("Чётные элементы: ");
        list.forEach(new ToDo<Integer>() {
            @Override
            public void toDo(Integer v) {
                if (v % 2 == 0) {
                    System.out.print(v + " ");
                }
            }
        });
        System.out.println();


        System.out.println("\n=== Сортировка рекурсивным слиянием ===");
        System.out.println("До сортировки:    " + list);
        list.sort();
        System.out.println("После сортировки: " + list);
        System.out.print("Обратный обход:   ");
        list.forEachReverse(v -> System.out.print(v + " "));
        System.out.println("\nУпорядочен: " + list.isSorted());


        System.out.println("\n=== Сортировка списка из 100000 случайных чисел ===");
        Random rnd = new Random(8);
        DoublyLinkedList big = new DoublyLinkedList();
        for (int i = 0; i < 100_000; i++) {
            big.addLast(rnd.nextInt(2_000_001) - 1_000_000);
```

```
        }
        int[] expected = big.toArray();
        Arrays.sort(expected);
        long t0 = System.nanoTime();
        big.sort();
        long t1 = System.nanoTime();
        System.out.printf("Время сортировки: %.1f мс%n", (t1 - t0) / 1e6);
        System.out.println("Упорядочен: " + big.isSorted());
        System.out.println("Совпадает с Arrays.sort: " + Arrays.equals(expected,
big.toArray()));

        System.out.println("\n=== Граничные случаи ===");
        DoublyLinkedList one = new DoublyLinkedList(5);
        one.sort();
        System.out.println("Сортировка из 1 элемента: " + one);
        System.out.println("remove(0) = " + one.remove(0) + ", список: " + one + ",
пуст: " + one.isEmpty());
        one.insert(0, 9);
        System.out.println("insert(0, 9) в пустой список: " + one);
        DoublyLinkedList dup = new DoublyLinkedList(3, 1, 3, 2, 1, 3);
        dup.sort();
        System.out.println("Сортировка с повторами: " + dup);


        System.out.println("\n=== Ошибки ===");
        try {
            list.get(100);
        } catch (IndexOutOfBoundsException e) {
            System.out.println("Ошибка: " + e.getMessage());
        }
        try {
            list.insert(-1, 5);
        } catch (IndexOutOfBoundsException e) {
            System.out.println("Ошибка: " + e.getMessage());
        }
        try {
            new DoublyLinkedList().remove(0);
        } catch (IndexOutOfBoundsException e) {
            System.out.println("Ошибка: " + e.getMessage());
        }


        list.clear();
        System.out.println("\nПосле clear(): " + list + ", размер = " +
list.size());
    }
}
```